SPE-229456-MS

Compressed Deep Learning for Drone-Based Visual Navigation

Noura H. Alhijri, Mustafa A. Al Ibrahim, and Pavel Golikov, EXPEC Advanced Research Center, Saudi Aramco

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.

Abstract

Visual navigation is a critical capability for drones in applications such as search and rescue, environmental monitoring, infrastructure inspection, and precision agriculture. However, real-time deployment on drones poses significant challenges due to limited computational resources, battery constraints, and the need for rapid decision-making. To address these challenges, this work focuses on designing and optimizing a lightweight deep learning model for predicting safe landing zones (SLZs) for a Seismic Acquisition Drone, enabling efficient onboard processing. We aim to predict SLZs by training a large SegFormer-B2 model for semantic segmentation. For this task, the model classifies each pixel into one of nine predefined classes. Based on this segmentation, the pipeline then generates a safe landing zone map. To enable real-time inference, we compress the trained SegFormer-B2 model using a knowledge distillation technique, transferring its knowledge to a smaller SegFormer-B0 model. SegFormer-B2 achieves slightly higher accuracy (95.37%) and average precision (99.22%) than SegFormer-B0 (94.78% accuracy and 98.79% precision). However, SegFormer-B0 offers significantly faster inference speed, processing 179 frames per second (FPS) on a GPU, compared to only 112 FPS for SegFormer-B2. This trade-off highlights SegFormer-B0 as a more practical and efficient choice for real-time applications where speed is crucial. These results demonstrate the effectiveness of knowledge distillation in reducing model size while preserving accuracy. Unlike existing SLZ methods that rely on high-compute drones, our approach enables fast, memory-efficient, and reliable predictions suitable for deployment on resource-constrained drones operating in complex environments.

Keywords: compressed model, seismic acquisition drone, neural network, machine learning, knowledge distillation

Introduction

Deep learning has emerged as a transformative technology, demonstrating remarkable performance across a wide range of disciplines, particularly in the era of big data. As the volume of data continues to grow exponentially, deep learning models are increasingly capable of capturing complex patterns and making accurate predictions in domains such as computer vision, natural language processing, healthcare, autonomous systems, and more. This impressive progress can be attributed not only to the availability of large datasets but also to continuous advancements in neural network architectures and hardware acceleration technologies, including powerful GPUs. Over the past decade, the design of deep learning models has undergone significant evolution. Initially, traditional feedforward neural networks were used for simple tasks. However, the field witnessed a major breakthrough with the introduction of Convolutional Neural Networks (CNNs), which revolutionized visual understanding by incorporating spatial hierarchies in their design. Popular CNN architectures such as AlexNet [1], VGG [2], ResNet [3], DenseNet [4], and MobileNet [5] paved the way for high-performance image classification, object detection, and segmentation tasks. These models introduced concepts like residual connections, dense connectivity, and depth-wise separable convolutions to improve learning efficiency and accuracy. More recently, the introduction of Transformer-based models has marked a paradigm shift in deep learning. Initially developed for natural language processing, the attention mechanism [6] allowed models to focus selectively on relevant parts of the input. This innovation led to powerful models such as BERT [7], GPT [8], and Vision Transformers (ViT) [9], which have set new performance benchmarks in both language and vision tasks.

Despite the remarkable advancements in deep learning, the computational demands associated with training and deploying these models in real-world applications have increased substantially. As neural networks grow deeper and more complex to improve performance across various domains, the requirements for processing power, memory, and storage have also escalated. This has created significant challenges, particularly in scenarios where resources are limited, such as in edge devices, real-time systems, and mobile applications, including drones and autonomous vehicles. To address these limitations, the development of efficient and lightweight deep learning models has become a critical area of research. Model compression techniques aim to reduce the size, complexity, and computational overhead of neural networks without significantly compromising their accuracy. These methods are essential for enabling the deployment of deep learning solutions on hardware-constrained environments, where power efficiency and low-latency inference are crucial. Among the most notable contributions in this field are knowledge distillation [10], pruning [11], and quantization [12]. Knowledge distillation involves training a smaller model to mimic the behavior of a larger, pre-trained model, thereby retaining much of the original performance while reducing model size. Pruning techniques selectively remove redundant or less significant weights and neurons from the network to decrease the number of parameters and computations. Quantization, on the other hand, reduces the precision of the model's parameters, converting them from large floating-point values to lower-bit representations, which significantly reduces memory usage and speeds up inference. Collectively, these techniques contribute to making deep learning more scalable, accessible, and practical for deployment in everyday devices and applications. Continued research in this area is essential to ensure that the benefits of deep learning can be harnessed efficiently across a wide range of platforms.

Methodology

Our proposed workflow utilizes real-time images captured by a drone-mounted camera to identify suitable landing zones. These images are processed using a lightweight semantic segmentation model capable of efficiently distinguishing between safe and non-safe landing areas. The overall method follows a multistep approach specifically designed to ensure high- confidence identification of safe landing zones for the Acquisition of Seismic Drone. We start by generating a simulated desert dataset tailored to the characteristics of the drone's operational environment. Then, this dataset is used to train a semantic segmentation model that performs pixel-wise classification across nine distinct classes: sand-land, grass-land, wall, building, movable-object, road, sand-road, obstacles, and non-flat terrain. Once the segmentation is complete, we apply a binary classification to differentiate between safe and non-safe zones. In this context, only sand-land and grass-land are considered safe, while all other classes are classified as non-safe. During inference, the drone captures live images and processes them through the trained model, which segments the image and labels each pixel accordingly. The final output allows the drone to assess its surroundings and make informed, real-time landing decisions with high reliability. This workflow is illustrated in Figure 1 demonstrates how our approach enables intelligent and autonomous landing for Seismic Acquisition Drones operating in desert environments.

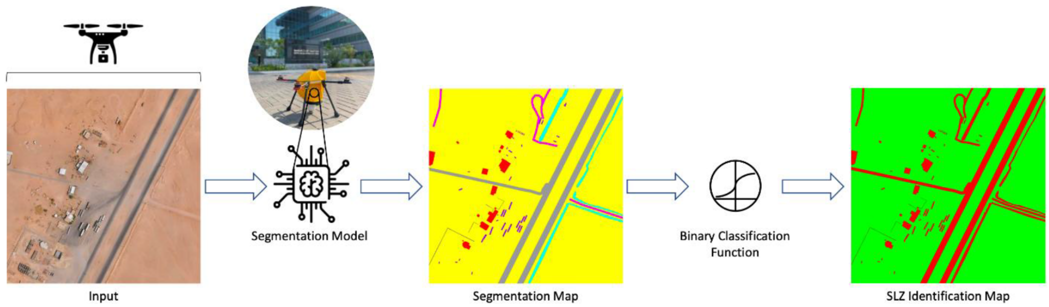

Figure 1—Our proposed workflow during inference: the input image is captured by the drone, processed by the lightweight SegFormer model for class segmentation, and then passed to a binary classification function to identify safe landing zones.

Simulation Framework and Dataset Generation

A high-quality dataset plays a critical role in the successful training of a deep learning model [13]. Such a dataset can effectively capture real-world complexity by incorporating domain-specific knowledge, while also increasing both the size of the dataset and the diversity within each class. In this work, we are provided with a single pair of a super-resolution image (~10Kx10K), captured over a desert by a seismic drone such as [14], along with its corresponding segmentation map that contains nine semantic classes. Our ultimate goal is to construct a comprehensive and high-quality dataset by utilizing this image in conjunction with its segmentation map. To achieve this, we randomly sample patches from various locations and altitudes within the image, along with their corresponding labeled segmentation maps, until we generate a total of 20,000 image samples. Each sample has a spatial resolution of 512x512 pixels. Variation in altitude between sampled patches improves the robustness of the dataset, making it less sensitive to drone height and more adaptable to diverse operating conditions during inference.

Segmentation Model Development and Training

Image segmentation is a fundamental computer vision technique that aims to classify each pixel within an image according to its semantic class. This task is particularly important in applications that require scene understanding, such as autonomous navigation and precision landing. For our segmentation task, we train the SegFormer neural network, a modern architecture inspired by Transformers [15]. Despite its lightweight design, which is achieved through the use of computationally efficient modules, SegFormer demonstrates superior performance compared to many traditional segmentation models. This makes it especially suitable for deployment on edge devices with limited computational resources, such as drones.

Our SegFormer model consists primarily of two main components: an encoder and a decoder. The encoder is responsible for downsampling the input image to extract rich feature representations using a sequence of Transformer blocks. These blocks utilize the self-attention mechanism, which effectively captures long-range dependencies in the image while operating on a reduced representation of the key sequence. The decoder, on the other hand, upsamples the extracted features to match the original spatial dimensions of the input image. Instead of traditional deconvolutional layers, the decoder employs multilayer perceptrons (MLPs), which contribute to faster inference and lower computational cost. Additionally, the SegFormer model architecture omits positional encoding, further enhancing its efficiency and lightweight design. Among the various versions of the SegFormer, we utilize the version B2, which contains around 27.5 million parameters, making it ideal for real-time drone-based semantic segmentation.

Model Compression

Deep learning models have demonstrated exceptional performance in complex tasks such as semantic segmentation, particularly when dealing with large-scale datasets. However, despite their accuracy and effectiveness, these models often come with significant operational and storage costs, making them less suitable for deployment on resource-constrained devices such as drones. In modern applications, model compression has become increasingly important, not only to reduce the size of large models but also to make them efficient enough to run in real time on edge devices with limited computational power.

To address this challenge, we applied knowledge distillation to compress the SegFormer-B2 model. This technique significantly reduces the number of parameters, which in turn improves inference time and optimizes memory usage on edge devices, while preserving the accuracy of the model. Specifically, in our work, we compress the original SegFormer- B2 model, which contains approximately 27.5 million parameters, down to the lightweight SegFormer-B0 model, which has approximately 3.7 million parameters. This substantial reduction results in a faster and more efficient deployment without a significant trade-off in performance.

As illustrated in Figure 2, the compression process involves using SegFormer-B2 as the pre-trained teacher model and SegFormer-B0 as the student model. During training, an input image is passed through the teacher model to generate soft probabilities across all classes. These soft targets are then used to contribute to the guide of the student model, forming a soft loss that captures the relationships between classes, going beyond traditional hard-label supervision. This distilled knowledge enables the student model to closely mimic the teacher's performance while being far more efficient.

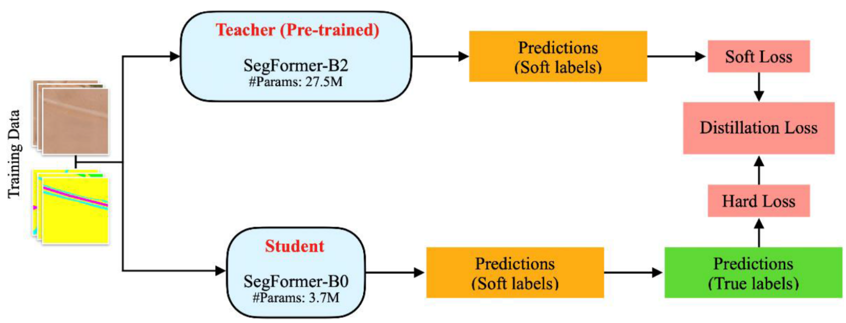

Figure 2—Knowledge distillation framework illustrating the training process of the SegFormer-B0 student model, which learns by distilling knowledge from the pretrained SegFormer-B2 teacher model.

Safe Landing Zone Identification

As mentioned earlier, accurately predicting safe landing zones with high confidence is crucial, particularly for autonomous drone applications where safety is a top priority. For this purpose, let us define S as the safe class and N as the non-safe class. Let I be a binary classification function that identifies safe landing zones based on the segmentation results. Refer to Table 1 for class definitions.

Table 1—Mapping between safe landing zone classes and segmentation classes.

Safe Landing Zones Classes	Segmentation Classes
S = safe	sand-land, grass-land
N = non-safe	wall, building, movable-object, road, sand-road, obstacles, non-flat terrain

We can formulate the probability of predicting a safe landing zone as follows:

$$p(SLZ) = p(class = S) = p(class = "sand - land") \cup p(class = "grass - land")$$

Since these two classes are mutually exclusive, the union simplifies to a summation.

$$p(SLZ) = p(class = "sand - land") + p(class = "grass - land") \tag{1}$$

However, relying solely on this probability may lead to false positives (FP), where a non-safe area is incorrectly classified as safe, a critical failure scenario that we strive to prevent. To mitigate this risk and enhance reliability, we introduce a high- confidence threshold:

$$I(SLZ) = \begin{cases} 1, & p(SLZ) \geq 0.99, \\ 0, & otherwise. \end{cases} \tag{2}$$

Experiments

We trained our models using four GPUs, each equipped with approximately 82 GB of memory. For both teacher and student models, training was conducted using a batch size of 8 and an input resolution of 256x256. The Adam optimizer was used for optimization, with a learning rate set to $5\text{x}10^{-4}$. The dataset was randomly initialized and then split into 80% for training and 20% for validation. We began by training the teacher model, which is more complex and is capable of capturing richer feature representations. Once the teacher model completed its training, we froze its weights to preserve its learned knowledge. Subsequently, we trained the student model by applying knowledge distillation, transferring insights from the teacher to improve the student's performance while maintaining a smaller and more efficient architecture.

Drone Simulation Setup

To validate the proposed workflow, we design an algorithm (see Algorithm 1) to simulate drone movement on a pre-defined simulation begins by specifying a set of *n* target points. Each point is defined by an *(x, y, z)* coordinate, representing the center location and altitude of the viewing window of a frame. The drone moves sequentially from the first point to the second, continuing until it reaches the final point, and then returns to the starting position to complete the loop. As the drone moves, a windowed patch is dynamically sampled from the larger scene. This sampling process accounts for the ratio between the window size and the drone's altitude to maintain spatial accuracy. The algorithm ensures smooth, continuous motion while preserving a consistent Field of View (FOV).

Algorithm 1: Moving Drone

Algorithm 1 Moving Drone

```
Input:
tensor: Input tensor from which patches are sampled
Xs: Tensor of sequential x-coordinates of n points to sample
Ys: Tensor of sequential y-coordinates of n points to sample
Hs: Tensor of n sequential scale parameters for each point
view range: Determines the size of the view or zoom level.
frame_size: Size of the output frames after resizing.
Output:
all_frames: List of resized frames sampled from the tensor
procedure Sample_Patches
    for i = 0 to n do
        row_length = view_range * (Hs[i] / tensor.Height)
        column_length = view_range * (Hs[i] / tensor.Width)
        start_row = Ys[i] - (row_length / 2)
        start_column = Xs[i] - (column_length / 2)
        end_row = start_row + row_length
        end_column = start_column + column_column
        frame = tensor[start_row:end_row, start_column:end_column]
        resized_frame = resize frame to (frame_size, frame_size)
        append resized frame to all_frames
    end for
    return all_frames
end procedure
```

Results

We evaluated and compared the performance of both models to highlight their effectiveness in terms of accuracy, speed, and confidence levels. The SegFormer-B0 model achieved an overall accuracy of 94.78%, an average precision of 98.79%, and an area under the ROC curve (AUROC) of 97.97%. In comparison, the larger SegFormer-B2 model reached a slightly higher accuracy of 95.37%, with an average precision of 99.22% and an AUROC of 98.55%. Although the performance difference between the two models is relatively small, the computational efficiency varies significantly. When deployed on a GPU, the SegFormer-B0 model processes at 179 frames per second (FPS), while SegFormer-B2 runs at 112 FPS on the same hardware.

Table 2—Comparison between SegFormer-B0 (student) and SegFormer-B2 (teacher) in terms of model size, classification metrics, and frames per second (FPS).

Model	# Param ↓	Accuracy ↑	Avg-Precision ↑	AUROC ↑	FPS(GPU) ↑	FPS(CPU) ↑
SegFormer-BO	3.7M	0.9478	0.9879	0.9797	179	0.5
SegFormer-B2	27.5M	0.9537	0.9922	0.9855	112	0.2

In terms of visual output (see Figure 3), the first column displays a sequence of frames captured by the live camera of the simulated drone as it moves over the scene. The second column illustrates the semantic segmentation predictions for each frame, identifying nine distinct classes, as shown in Figure 3. The third column presents the segmentation confidence levels, where most pixels exhibit high confidence values close to 1.0. However, pixels located at the edges of semantic objects, particularly between different classes, tend to have lower confidence scores, often around 0.5, due to color blending and ambiguity in class separation. The fourth column displays the calculated Safe Landing Zones (SLZ), classified into two categories as defined in the Figure 3. Finally, the fifth column visualizes the confidence levels associated with the SLZ predictions. In this view, higher levels of green intensity indicate greater confidence in areas that are safe for landing, offering an intuitive interpretation of drone decision-making in dynamic environments.

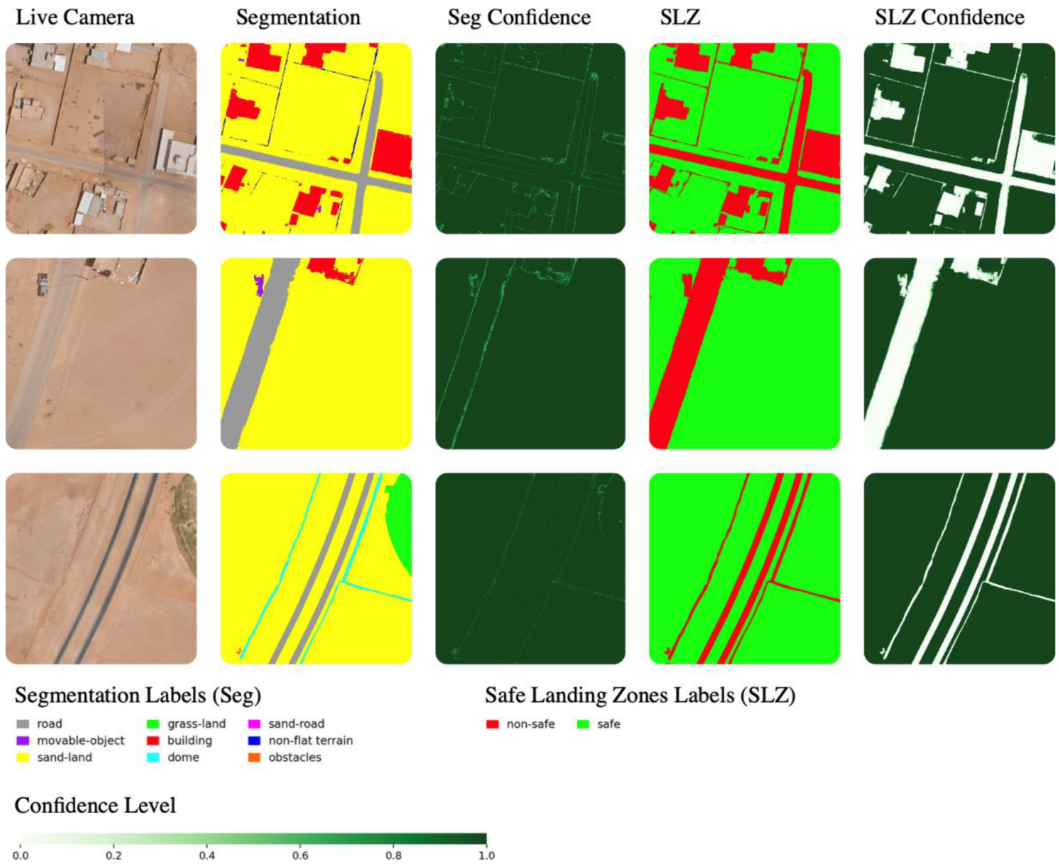

Figure 3—Illustrative examples of simulated drone inputs alongside their SegFormer-B0 segmentation outputs: (Live Camera) shows simulated frames captured by a moving drone's live camera; (Segmentation) presents the predicted semantic segmentation for each frame; (Seg Confidence) displays the confidence levels associated with the segmentation predictions; (SLZ) indicates the identified Safe Landing Zones derived from the segmentation results; and (SLZ Confidence) reflects the confidence level indicating the reliability of each detected SLZ.

Discussion

The substantial difference in inference speed highlights the advantage of the compressed model, the SegFormer-B0, especially in real-time applications like drone navigation. Despite having fewer parameters, SegFormer-B0 achieved competitive accuracy, making it a strong candidate for drone-based deployments. This aligns with previous studies showing that lightweight models can still perform effectively in edge environments.

The outputs generated by both models are highly comparable in terms of segmentation quality and safe landing zone identification. As illustrated in Figure 4, the predictions from SegFormer-B0 closely resemble those of SegFormer-B2 for the two examples, despite the significant reduction in model size from 27.5M to 3.7M and computational requirements. This visual comparison highlights the effectiveness of knowledge distillation in preserving performance while enabling lightweight deployment.

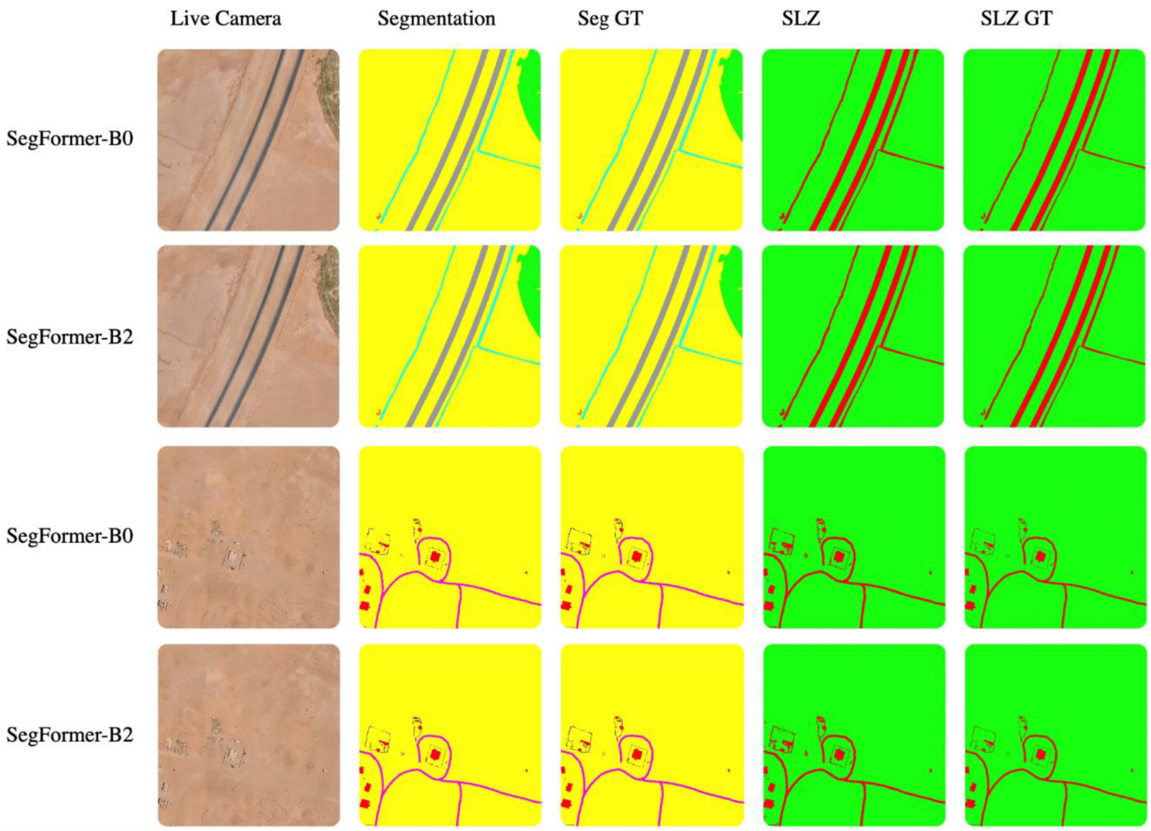

Figure 4—Visual comparison of segmentation outputs for two distinct camera frames, showcasing the performance differences between SegFormer-B0 and SegFormer-B2 models.

Conclusion

In this paper, we propose a compressed and lightweight semantic segmentation model specifically designed for deployment on a seismic acquisition drone used in land exploration missions. The primary objective of the model is to perform realtime semantic segmentation on live video frames captured by the drone's onboard camera. Each pixel in the incoming frames is classified into predefined classes to enable precise environmental understanding. Based on this pixel-level classification, the system is capable of identifying Safe Landing Zones (SLZs), allowing the drone to autonomously assess and select suitable areas for safe landing during operation. The proposed model is optimized for low-latency inference and minimal resource consumption, making it well-suited for edge devices with limited computational capabilities, such as drones. To achieve this, various model compression techniques were applied, including knowledge distillation and quantization, resulting in a significant reduction in model size while maintaining acceptable segmentation accuracy. Experimental results demonstrate that the model achieves real-time inference speed on resource-constrained hardware with high precision in SLZ detection, validating its practicality and effectiveness for the target application.

The main limitation of the current models lies in their lack of stability during drone movement. These models are not specifically designed for continuous video sequences, which reduces the consistency of their predictions. For instance, when the drone shifts slightly, consecutive frames may share overlapping regions, yet the model's predictions for these frames can differ significantly. Future work will focus on extending the model's capabilities by training on continuous video sequences rather than static frames, thereby enabling better temporal consistency and smoother segmentation results over time. Furthermore, enhancements will be introduced to improve the robustness and reliability of SLZ identification under varying environmental conditions, such as complex terrain. In conclusion, the proposed model successfully demonstrates the feasibility of deploying semantic segmentation on-board drones for real-time SLZ detection. It provides a practical solution that balances accuracy and efficiency, paving the way for safer and more autonomous drone-based seismic exploration in complex and remote environments.

References

1. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Adv. Neural Inf. Process. Syst.*, vol. **25**, 2012.
2. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *CoRR*, vol. **abs/1409.1556**, 2014, [Online]. Available: https://api.semanticscholar.org/CorpusID:14124313
3. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.
4. G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," in 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 2261–2269. doi: 10.1109/CVPR.2017.243.
5. A. G. Howard et al, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," ArXiv Prepr. ArXiv170404861, 2017.
6. A. Vaswani et al, "Attention is all you need," *Adv. Neural Inf. Process. Syst.*, vol. **30**, 2017.
7. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers), 2019, pp. 4171–4186.
8. A. Radford and K. Narasimhan, "Improving Language Understanding by Generative Pre-Training," 2018. [Online]. Available: https://api.semanticscholar.org/CorpusID:49313245
9. A. Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021.
10. G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," ArXiv Prepr. ArXiv150302531, 2015.
11. P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference" ArXiv Prepr. ArXiv161106440, 2016.
12. B. Jacob et al, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 2704–2713.
13. Y. Gong, G. Liu, Y. Xue, R. Li, and L. Meng, "A survey on dataset quality in machine learning," *Inf. Softw. Technol.*, vol. **162**, p. 107268, 2023.
14. E. Alfataierge, P. Golikov, A. Ramdani, and A. Alshuhail, "Advancing autonomous seismic data acquisition with robotic solutions," in SEG International Exposition and Annual Meeting, SEG, 2024, p. SEG-2024.
15. E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: simple and efficient design for semantic segmentation with transformers," in Proceedings of the 35th International Conference on Neural Information Processing Systems, in NIPS ′21. Red Hook, NY, USA: Curran Associates Inc., 2021.